

SIT233 Final Project Report

Nirosh Ravindran

¹Department of Cybersecurity

BSCP|CS|63|134

rnirosh134@cicracampus.net

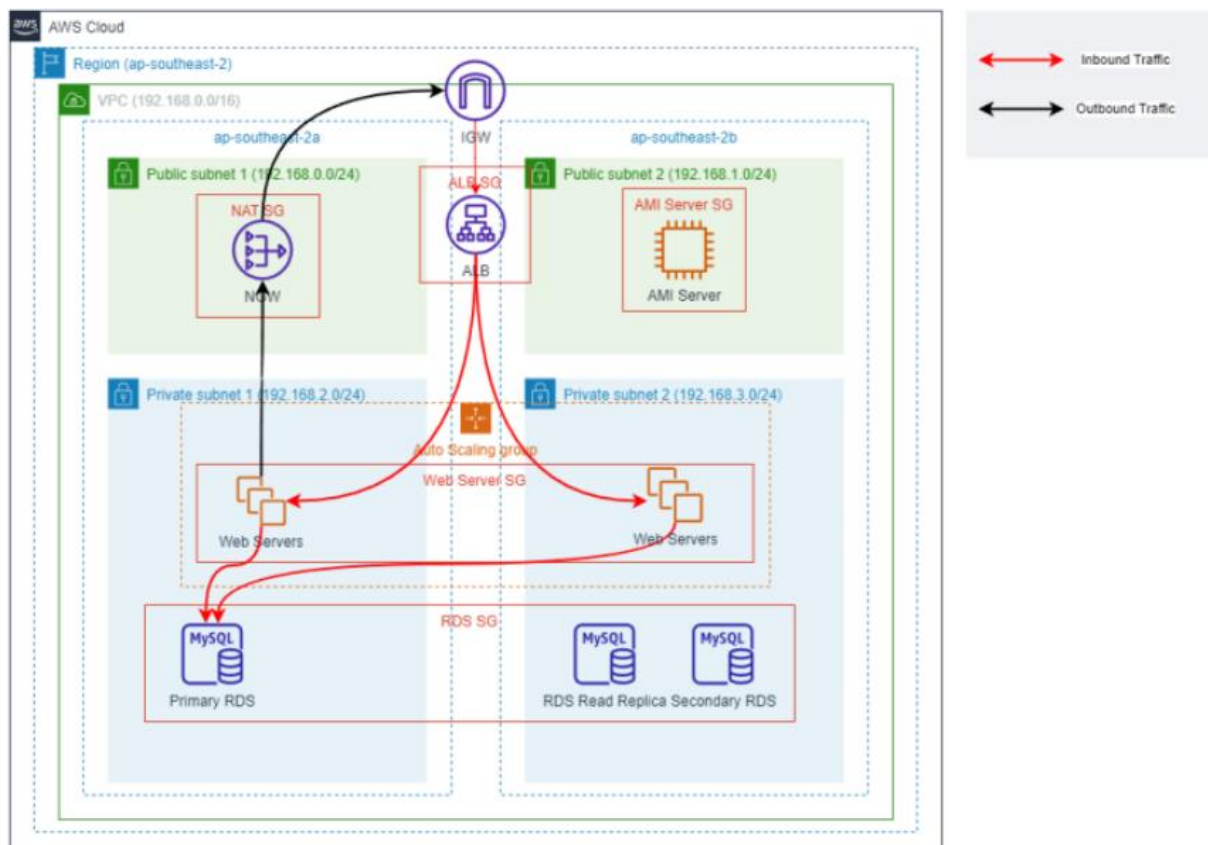
Abstract— This document offers a thorough account of the procedures followed in order to set up an AWS infrastructure for WordPress website hosting. A VPC, subnets, an internet gateway, and route tables are all created during the procedure. However, the project was unable to proceed past step 7 due to a problem with the formation of an IAM role. This paper describes the problems that were faced and the steps that were successful.

Keywords— AWS, VPC, Subnets, Internet Gateway, Route Tables, IAM Role, WordPress Hosting

I. INTRODUCTION

The method of setting up an AWS infrastructure to host a WordPress website is reported in this document. A Virtual Private Cloud (VPC), subnet configuration, internet gateway setup, and route table configuration are all part of the setup process. Sadly, a problem that arose during the creation of an IAM position forced the project to be suspended and prevented it from being completed.

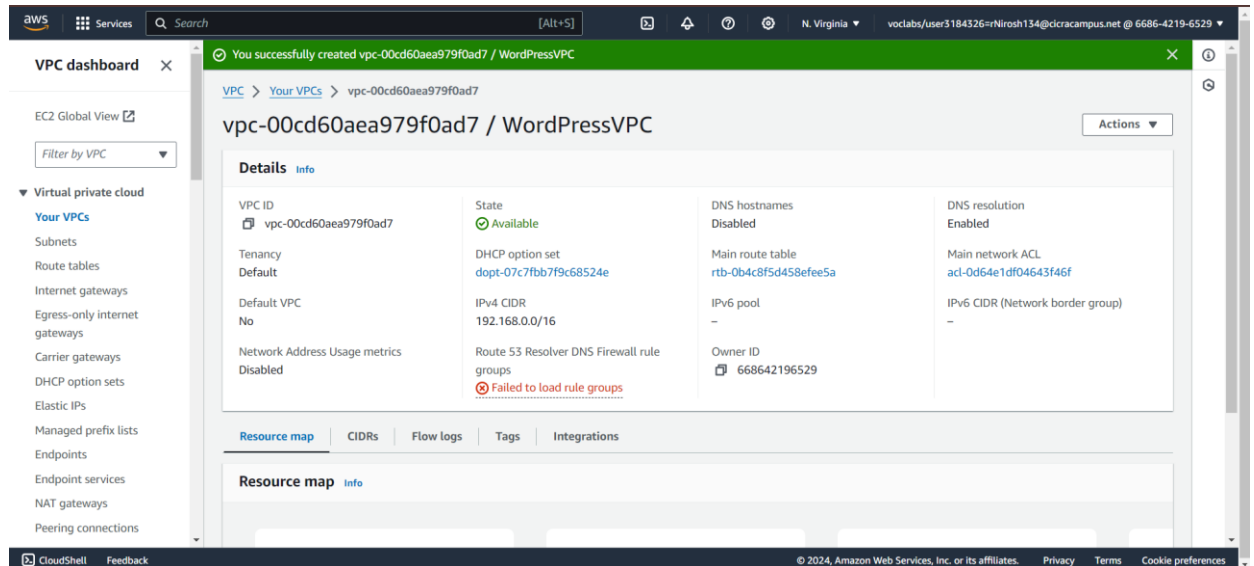
II. DESIGN DIAGRAM



III. AWS INFRASTRUCTURE SETUP

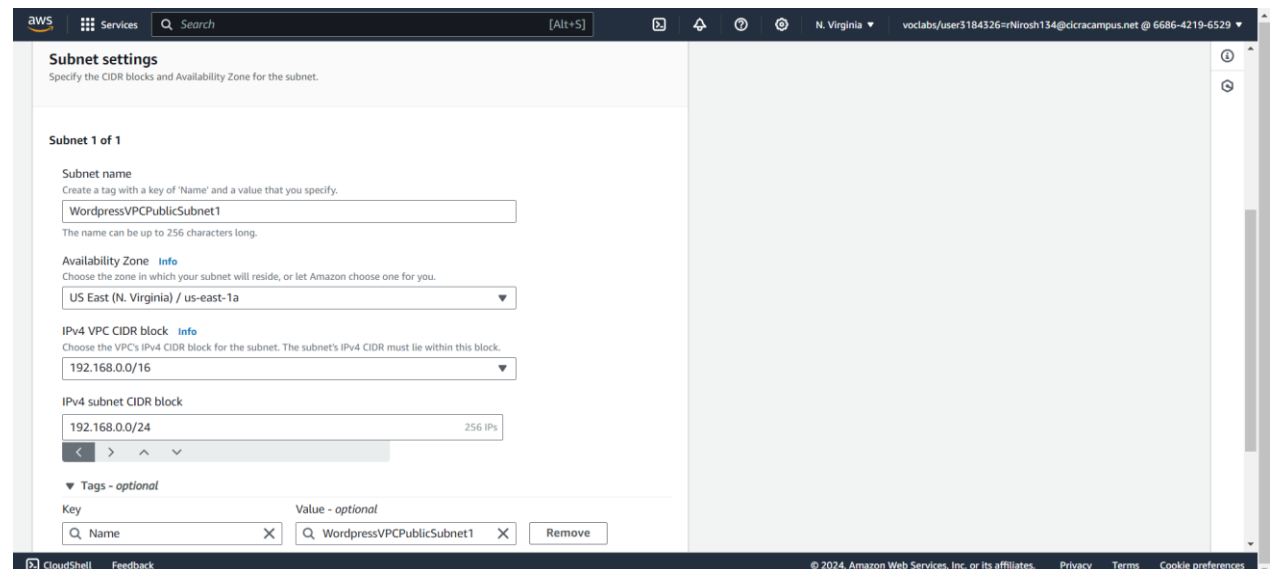
A. Create VPC

I started by navigating to the AWS Management Console's VPC Dashboard. I choose the VPC option from the Services menu in order to open the VPC Dashboard. I choose "Create VPC" after clicking on "Your VPCs" on the left navigation pane to start a new VPC. I completed the following fields on the VPC creation form: I left the tenancy as "Default," gave the VPC the name "WordPressVPC," and changed the IPv4 CIDR block to "10.0.0.0/16." At last, I selected "Create VPC" to finish the creation procedure.



B. Create Subnets

I began by selecting "Subnets" from the left navigation pane in order to configure the subnets. I chose "Create subnet" and filled in the following information to create the first public subnet: I gave it the name "PublicSubnet1", selected "WordPressVPC" as the virtual private cloud (VPC), "us-east-1a" as the availability zone, and "10.0.0.0/24" as the IPv4 CIDR block. The second public subnet was created using the same procedure; it was given the name "PublicSubnet2", a different availability zone (such as "us-east-1b"), and a CIDR block of "10.0.1.0/24". I took comparable actions for the private subnets. With "10.0.2.0/24" in "us-east-1a" and "10.0.3.0/24" in "us-east-1b," I established "PrivateSubnet1" and "PrivateSubnet2".



The screenshot shows the AWS VPC dashboard with a green notification banner at the top stating: "You have successfully created 1 subnet: subnet-0ed6c0afd6341206b". Below the banner, the "Subnets (1) Info" section is visible, featuring a search bar and a "Clear filters" button. A table lists the created subnet:

Name	Subnet ID	State	VPC	IPv4 CIDR
WordpressVPCPublicSubnet1	subnet-0ed6c0afd6341206b	Available	vpc-00cd60aea979f0ad7 Wor...	192.168.0.0/24

The left sidebar shows the "Virtual private cloud" section with links to "Your VPCs", "Subnets", "Route tables", "Internet gateways", and "Egress-only internet".

This screenshot shows the AWS VPC dashboard after creating a second subnet. The notification banner reads: "You have successfully created 1 subnet: subnet-0addfa6e884698f34". The "Subnets (1) Info" section displays the following table:

Name	Subnet ID	State	VPC	IPv4 CIDR
WordpressVPCPublicSubnet2	subnet-0addfa6e884698f34	Available	vpc-00cd60aea979f0ad7 Wor...	192.168.1.0/24

The interface is consistent with the previous screenshot, showing the same sidebar and dashboard layout.

The screenshot displays the "Subnet settings" page for "Subnet 1 of 1". It includes the following configuration details:

- Subnet name:** WordpressVPCPrivateSubnet1
- Availability Zone:** US East (N. Virginia) / us-east-1a
- IPv4 VPC CIDR block:** 192.168.0.0/16
- IPv4 subnet CIDR block:** 192.168.2.0/24 (256 IPs)
- Tags - optional:** A table with one tag: Key: Name, Value: WordpressVPCPrivateSubnet1.

The bottom of the page shows the "CloudShell" and "Feedback" links, along with the copyright notice: "© 2024 Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences".

This screenshot shows the AWS VPC dashboard after creating a third subnet. The notification banner states: "You have successfully created 1 subnet: subnet-0abdd97fba3a0e134". The "Subnets (1) Info" section shows the following table:

Name	Subnet ID	State	VPC	IPv4 CIDR
WordpressVPCPrivateSubnet1	subnet-0abdd97fba3a0e134	Available	vpc-00cd60aea979f0ad7 Wor...	192.168.2.0/24

The dashboard layout, including the sidebar and search filters, remains the same as in the previous screenshots.

Subnet settings
Specify the CIDR blocks and Availability Zone for the subnet.

Subnet 1 of 1

Subnet name
Create a tag with a key of 'Name' and a value that you specify.
WordpressVPCPrivateSubnet2
The name can be up to 256 characters long.

Availability Zone [Info](#)
Choose the zone in which your subnet will reside, or let Amazon choose one for you.
US East (N. Virginia) / us-east-1b

IPv4 VPC CIDR block [Info](#)
Choose the VPC's IPv4 CIDR block for the subnet. The subnet's IPv4 CIDR must lie within this block.
192.168.0.0/16

IPv4 subnet CIDR block
192.168.3.0/24 256 IPs

Tags - optional

Key	Value - optional
Name	WordpressVPCPrivateSubnet2

Remove

VPC dashboard

EC2 Global View [Filter by VPC](#)

Subnets (10) [Info](#)

Find resources by attribute or tag

Subnet ID : subnet-0f78cbde01a3e4c24 Clear filters

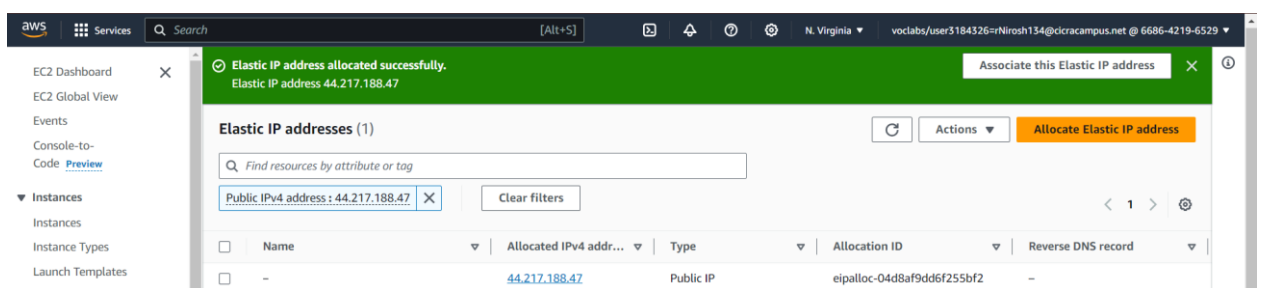
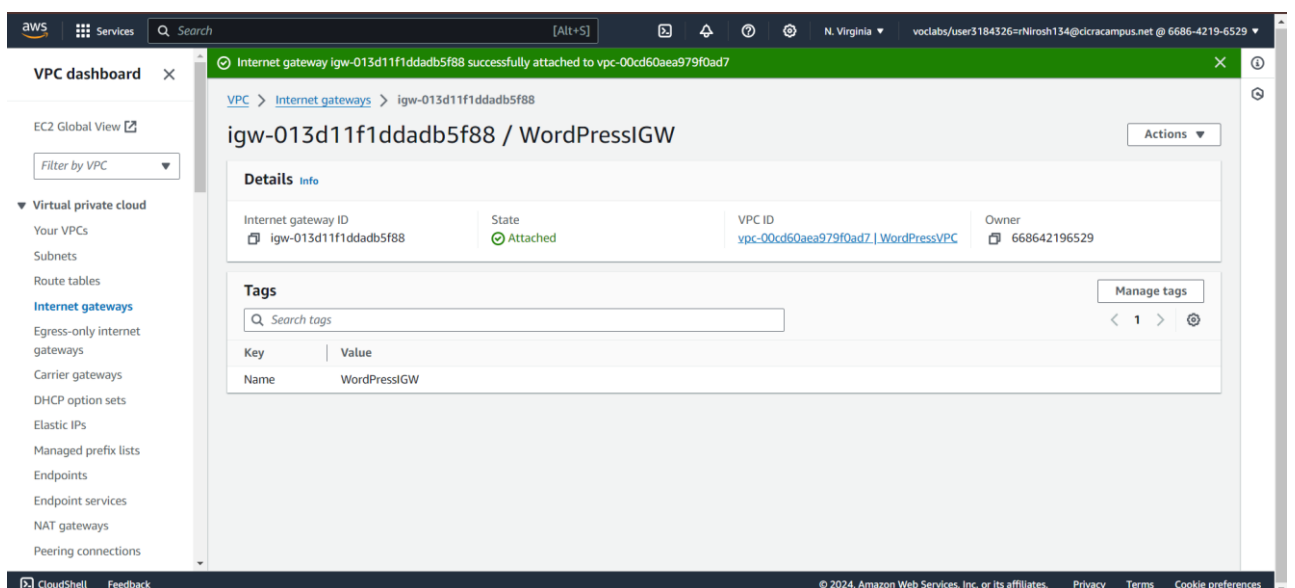
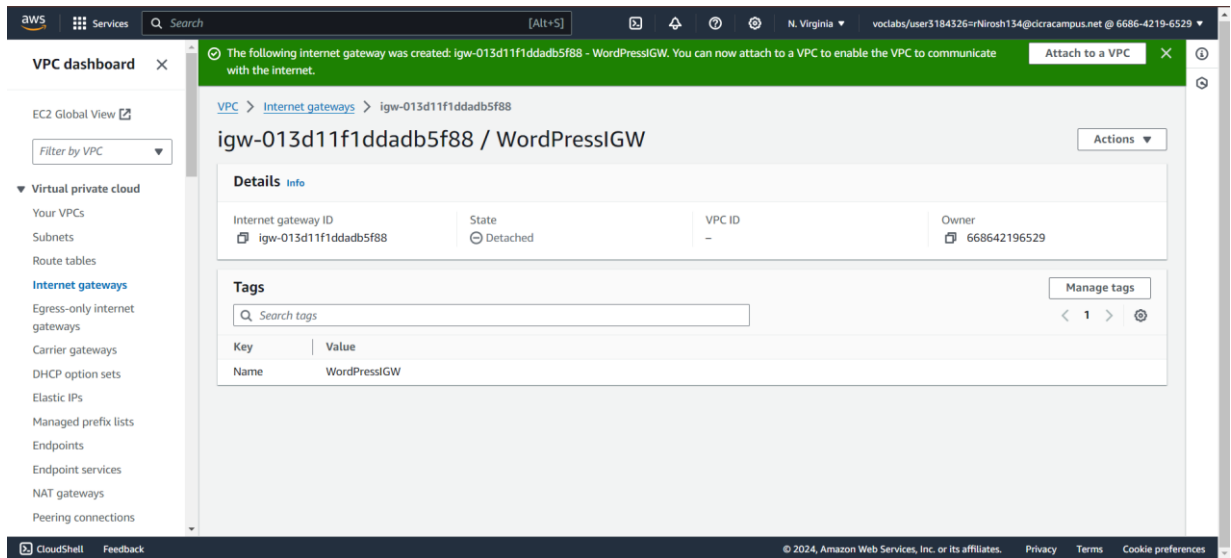
Name	Subnet ID	State	VPC	IPv4 CIDR
WordpressVPCPrivateSubnet1	subnet-0abdd97fba3a0e134	Available	vpc-00cd60aea979f0ad7 Wor...	192.168.2.0/24
-	subnet-03cd4ea5bcad9f1c5	Available	vpc-0a766665f0949b44c	172.31.64.0/20
-	subnet-0651d16276c148f09	Available	vpc-0a766665f0949b44c	172.31.0.0/20
WordpressVPCPublicSubnet2	subnet-0addfa6e884698f34	Available	vpc-00cd60aea979f0ad7 Wor...	192.168.1.0/24
-	subnet-085adfa5373acd1b3	Available	vpc-0a766665f0949b44c	172.31.16.0/20
WordpressVPCPublicSubnet1	subnet-00ed6c0afdc6341206b	Available	vpc-00cd60aea979f0ad7 Wor...	192.168.0.0/24
-	subnet-0145c94905e071a75	Available	vpc-0a766665f0949b44c	172.31.32.0/20
-	subnet-03dabaa6ca0e9790b	Available	vpc-0a766665f0949b44c	172.31.80.0/20
-	subnet-07e21088b356ba4d8	Available	vpc-0a766665f0949b44c	172.31.48.0/20
WordpressVPCPrivateSubnet2	subnet-0f78cbde01a3e4c24	Available	vpc-00cd60aea979f0ad7 Wor...	192.168.3.0/24

Select a subnet

C. Create Internet Gateway

In order to facilitate communication between the VPC and the internet, I then developed an Internet Gateway. I selected "Create internet gateway" after selecting "Internet Gateways" in the left navigation pane. I gave it the name "WordPressIGW" and continued by selecting "Create internet gateway". I chose "WordPressIGW,"

clicked "Actions," selected "Attach to VPC," choose "WordPressVPC," and then I clicked "Attach internet gateway" to confirm that I wanted to attach this internet gateway to the VPC.



D. Create NAT Gateway

To access the internet through my private subnets, I required a NAT Gateway. Making an Elastic IP was my first step. I selected "Allocate Elastic IP address" and then "Allocate" after going to the "Elastic IPs" section of the EC2 Dashboard. I went back to the VPC Dashboard after setting up the Elastic IP, clicked on "NAT Gateways" on the left navigation pane, and chose "Create NAT Gateway". I picked the newly established Elastic IP, PublicSubnet1 as the subnet, and clicked "Create NAT Gateway".

The image consists of two screenshots from the AWS Management Console. The top screenshot shows the "Create NAT gateway" page. The "NAT gateway settings" section includes:

- Name - optional:** A text box containing "NATpublic1".
- Subnet:** A dropdown menu showing "subnet-0ed6c0afd6341206b (WordpressVPCPublicSubnet1)".
- Connectivity type:** Radio buttons for "Public" (selected) and "Private".
- Elastic IP allocation ID:** A dropdown menu showing "eipalloc-04d8af9dd6f255bf2" with an "Allocate Elastic IP" button next to it.

 The bottom screenshot shows the "Details" page for the NAT gateway "nat-0299cf2ec72fa83f6 / NATpublic1". The "Details" section includes:

- NAT gateway ID:** nat-0299cf2ec72fa83f6
- NAT gateway ARN:** arn:aws:ec2:us-east-1:668642196529:natgateway/nat-0299cf2ec72fa83f6
- VPC:** vpc-00cd60aea979f0ad7 / WordpressVPC
- Connectivity type:** Public
- Primary public IPv4 address:** 44.217.188.47
- Subnet:** subnet-0ed6c0afd6341206b / WordpressVPCPublicSubnet1
- State:** Available
- Primary private IPv4 address:** 192.168.0.32
- Created:** Friday, June 28, 2024 at 22:04:25 GMT+5:30
- State message:** -
- Primary network interface ID:** eni-040f81f8a60a428a4
- Deleted:** -

 Below the details, there is a section for "Secondary IPv4 addresses" with a search bar and a table with columns: Private IPv4 address, Allocation ID, Association ID, and Public IPv4 address.

E. Create Route Tables

I developed a route table for the public subnets in order to control traffic routing within the VPC. I selected "Create route table" after going to "Route Tables" in the left navigation pane. With "PublicRouteTable" as its name, I chose "WordpressVPC" as the VPC. I edited the routes by choosing "PublicRouteTable," selecting the "Routes" tab, and then choosing "Edit routes" after generating the route table. I created a route and saved

it with the destination set to "0.0.0.0/0" and the target set to "WordPressIGW". Lastly, I chose "PublicRouteTable," clicked on the "Subnet Associations" tab, chose "Edit subnet associations," picked "PublicSubnet1" and "PublicSubnet2," and saved the associations in order to associate the public subnets with this route table.

The screenshot shows the 'Create route table' page in the AWS Management Console. The page has a dark header with the AWS logo, 'Services' menu, a search bar, and user information. The breadcrumb trail is 'VPC > Route tables > Create route table'. The main heading is 'Create route table' with an 'Info' link. Below this is a descriptive sentence: 'A route table specifies how packets are forwarded between the subnets within your VPC, the internet, and your VPN connection.'

The 'Route table settings' section contains two fields: 'Name - optional' with the value 'PublicRouteTable' and 'VPC' with a dropdown menu showing 'vpc-00cd60aea979f0ad7 (WordPressVPC)'. The 'Tags' section has a table with columns 'Key' and 'Value - optional'. It contains one tag with Key 'Name' and Value 'PublicRouteTable'. There is an 'Add new tag' button and a note 'You can add 49 more tags.'

The footer includes 'CloudShell', 'Feedback', a copyright notice '© 2024, Amazon Web Services, Inc. or its affiliates.', and links for 'Privacy', 'Terms', and 'Cookie preferences'.

The screenshot shows the 'Route table details' page for 'rtb-061ab8b9fbee497e'. A green success message at the top states 'Route table rtb-061ab8b9fbee497e | PublicRouteTable was created successfully.' The breadcrumb trail is 'VPC > Route tables > rtb-061ab8b9fbee497e'. The main heading is 'rtb-061ab8b9fbee497e / PublicRouteTable' with an 'Actions' dropdown.

The 'Details' section shows: 'Route table ID' as 'rtb-061ab8b9fbee497e', 'Main' as 'No', 'Explicit subnet associations' as '-', and 'Edge associations' as '-'. It also shows 'VPC' as 'vpc-00cd60aea979f0ad7 | WordPressVPC' and 'Owner ID' as '668642196529'.

The 'Routes' tab is selected, showing a table with one route. The table has columns: 'Destination', 'Target', 'Status', and 'Propagated'. The route has Destination '192.168.0.0/16', Target 'local', Status 'Active' (with a green checkmark), and Propagated 'No'.

The footer is identical to the previous screenshot.

The screenshot shows the 'Edit routes' page for the route table 'rtb-061ab8b9fbee497e'. The breadcrumb trail is 'VPC > Route tables > rtb-061ab8b9fbee497e > Edit routes'. The main heading is 'Edit routes'.

The page displays a table for editing routes with columns: 'Destination', 'Target', 'Status', and 'Propagated'. The first row shows Destination '192.168.0.0/16', Target 'local', Status 'Active', and Propagated 'No'. The second row shows Destination '0.0.0.0/0', Target 'Internet Gateway', Status '-', and Propagated 'No'. There is a 'Remove' button next to the second row.

Below the table is an 'Add route' button. At the bottom right are 'Cancel', 'Preview', and 'Save changes' buttons.

Updated routes for rtb-061ab8b9fbee497e / PublicRouteTable successfully

VPC > Route tables > rtb-061ab8b9fbee497e

rtb-061ab8b9fbee497e / PublicRouteTable

Actions

Details info

Route table ID rtb-061ab8b9fbee497e	Main No	Explicit subnet associations -	Edge associations -
VPC vpc-00cd60aea979f0ad7 WordPressVPC	Owner ID 668642196529		

Routes (2)

Filter routes

Destination	Target	Status	Propagated
0.0.0.0/0	igw-013d11f1ddadb5f88	Active	No
192.168.0.0/16	local	Active	No

CloudShell Feedback

© 2024, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

VPC > Route tables > rtb-061ab8b9fbee497e > Edit subnet associations

Edit subnet associations

Change which subnets are associated with this route table.

Available subnets (2/4)

Filter subnet associations

	Name	Subnet ID	IPv4 CIDR	IPv6 CIDR	Route table ID
☐	WordpressVPCPrivateSubnet1	subnet-0abdd97fba3a0e134	192.168.2.0/24	-	Main (rtb-0b4c8f5d458efee5a)
☒	WordpressVPCPublicSubnet2	subnet-0addfa6e884698f34	192.168.1.0/24	-	Main (rtb-0b4c8f5d458efee5a)
☒	WordpressVPCPublicSubnet1	subnet-0ed6c0afd6341206b	192.168.0.0/24	-	Main (rtb-0b4c8f5d458efee5a)
☐	WordpressVPCPrivateSubnet2	subnet-0f78cbde01a3e4c24	192.168.3.0/24	-	Main (rtb-0b4c8f5d458efee5a)

Selected subnets

subnet-0ed6c0afd6341206b / WordpressVPCPublicSubnet1 X subnet-0addfa6e884698f34 / WordpressVPCPublicSubnet2 X

Cancel Save associations

You have successfully updated subnet associations for rtb-061ab8b9fbee497e / PublicRouteTable.

VPC > Route tables > rtb-061ab8b9fbee497e

rtb-061ab8b9fbee497e / PublicRouteTable

Actions

Details info

Route table ID rtb-061ab8b9fbee497e	Main No	Explicit subnet associations 2 subnets	Edge associations -
VPC vpc-00cd60aea979f0ad7 WordPressVPC	Owner ID 668642196529		

Routes (2)

Filter routes

Destination	Target	Status	Propagated
0.0.0.0/0	igw-013d11f1ddadb5f88	Active	No

aws Services Search [Alt+S] N. Virginia voclabs/user3184326=rNirosh134@icracampus.net @ 6686-4219-6529

VPC > Route tables > Create route table

Create route table [Info](#)

A route table specifies how packets are forwarded between the subnets within your VPC, the internet, and your VPN connection.

Route table settings

Name - optional
Create a tag with a key of 'Name' and a value that you specify.

PrivateRouteTable

VPC
The VPC to use for this route table.

vpc-00cd60aea979f0ad7 (WordPressVPC)

Tags

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key **Value - optional**

Name PrivateRouteTable Remove

Add new tag

You can add 49 more tags.

CloudShell Feedback © 2024, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

aws Services Search [Alt+S] N. Virginia voclabs/user3184326=rNirosh134@icracampus.net @ 6686-4219-6529

VPC dashboard

EC2 Global View

Filter by VPC

Virtual private cloud

- Your VPCs
- Subnets
- Route tables
- Internet gateways
- Egress-only internet gateways
- Carrier gateways
- DHCP option sets
- Elastic IPs
- Managed prefix lists
- Endpoints
- Endpoint services
- NAT gateways
- Peering connections

Route table rtb-0a88230389eed406d | PrivateRouteTable was created successfully.

VPC > Route tables > rtb-0a88230389eed406d

rtb-0a88230389eed406d / PrivateRouteTable

Actions

Details [Info](#)

Route table ID	Main	Explicit subnet associations	Edge associations
rtb-0a88230389eed406d	No	-	-
VPC	Owner ID		
vpc-00cd60aea979f0ad7 WordPressVPC	668642196529		

Routes

Subnet associations | Edge associations | Route propagation | Tags

Routes (1)

Filter routes

Destination	Target	Status	Propagated
192.168.0.0/16	local	Active	No

CloudShell Feedback © 2024, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

aws Services Search [Alt+S] N. Virginia voclabs/user3184326=rNirosh134@icracampus.net @ 6686-4219-6529

VPC > Route tables > rtb-0a88230389eed406d > Edit subnet associations

Edit subnet associations

Change which subnets are associated with this route table.

Available subnets (2/4)

Filter subnet associations

	Name	Subnet ID	IPv4 CIDR	IPv6 CIDR	Route table ID
☒	WordpressVPCPrivateSubnet1	subnet-0abdd97fba3a0e134	192.168.2.0/24	-	Main (rtb-0b4c8f5d458fefe5a)
☐	WordpressVPCPublicSubnet2	subnet-0addfa6e884698f34	192.168.1.0/24	-	rtb-061ab8b9fbee497e / PublicRoute...
☐	WordpressVPCPublicSubnet1	subnet-0ed6c0afd6341206b	192.168.0.0/24	-	rtb-061ab8b9fbee497e / PublicRoute...
☒	WordpressVPCPrivateSubnet2	subnet-0f78cbde01a3e4c24	192.168.3.0/24	-	Main (rtb-0b4c8f5d458fefe5a)

Selected subnets

subnet-0abdd97fba3a0e134 / WordpressVPCPrivateSubnet1 subnet-0f78cbde01a3e4c24 / WordpressVPCPrivateSubnet2

Cancel Save associations

aws Services Search [Alt+S] N. Virginia voclabs/user3184326-rNirosh134@dcrcampus.net @ 6686-4219-6529

VPC dashboard ×

EC2 Global View [↗](#)

Filter by VPC ▾

▼ Virtual private cloud

- Your VPCs
- Subnets
- Route tables**
- Internet gateways
- Egress-only internet gateways
- Carrier gateways
- DHCP option sets
- Elastic IPs
- Managed prefix lists
- Endpoints
- Endpoint services
- NAT gateways
- Peering connections

CloudShell Feedback

© 2024, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

You have successfully updated subnet associations for rtb-0a88230389eed406d / PrivateRouteTable.

VPC > Route tables > rtb-0a88230389eed406d

rtb-0a88230389eed406d / PrivateRouteTable

Actions ▾

Details Info

Route table ID rtb-0a88230389eed406d	Main No	Explicit subnet associations 2 subnets	Edge associations -
VPC vpc-00cd60aea979f0ad7 WordPressVPC	Owner ID 668642196529		

Routes Subnet associations Edge associations Route propagation Tags

Routes (1) Both ▾ Edit routes

Filter routes

Destination	Target	Status	Propagated
192.168.0.0/16	local	Active	No

aws Services Search [Alt+S] N. Virginia voclabs/user3184326-rNirosh134@dcrcampus.net @ 6686-4219-6529

VPC > Route tables > rtb-0a88230389eed406d > Edit routes

Edit routes

Destination	Target	Status	Propagated
192.168.0.0/16	local	Active	No
0.0.0.0/0	NAT Gateway	-	No
	nat-0299cf2ec72fa83f6		

Add route

Cancel Preview **Save changes**

aws Services Search [Alt+S] N. Virginia voclabs/user3184326-rNirosh134@dcrcampus.net @ 6686-4219-6529

VPC dashboard ×

EC2 Global View [↗](#)

Filter by VPC ▾

▼ Virtual private cloud

- Your VPCs
- Subnets
- Route tables**
- Internet gateways
- Egress-only internet gateways
- Carrier gateways
- DHCP option sets
- Elastic IPs
- Managed prefix lists
- Endpoints
- Endpoint services
- NAT gateways
- Peering connections

CloudShell Feedback

© 2024, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Updated routes for rtb-0a88230389eed406d / PrivateRouteTable successfully

VPC > Route tables > rtb-0a88230389eed406d

rtb-0a88230389eed406d / PrivateRouteTable

Actions ▾

Details Info

Route table ID rtb-0a88230389eed406d	Main No	Explicit subnet associations 2 subnets	Edge associations -
VPC vpc-00cd60aea979f0ad7 WordPressVPC	Owner ID 668642196529		

Routes Subnet associations Edge associations Route propagation Tags

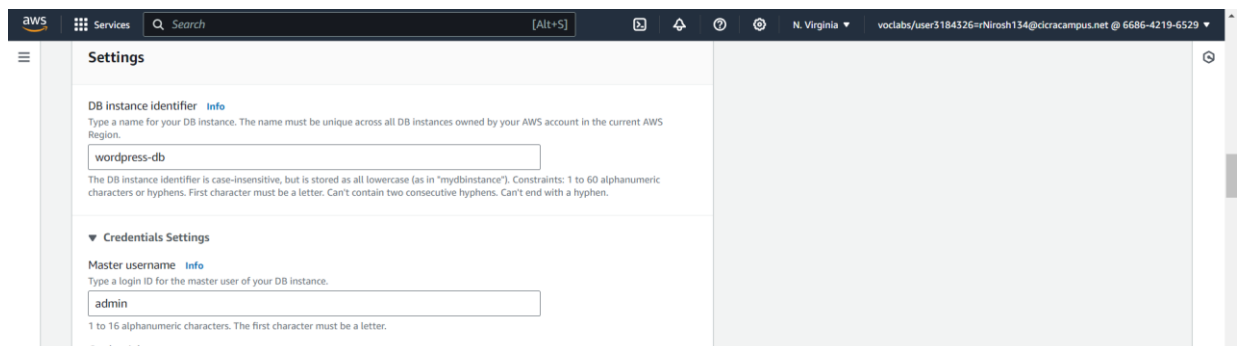
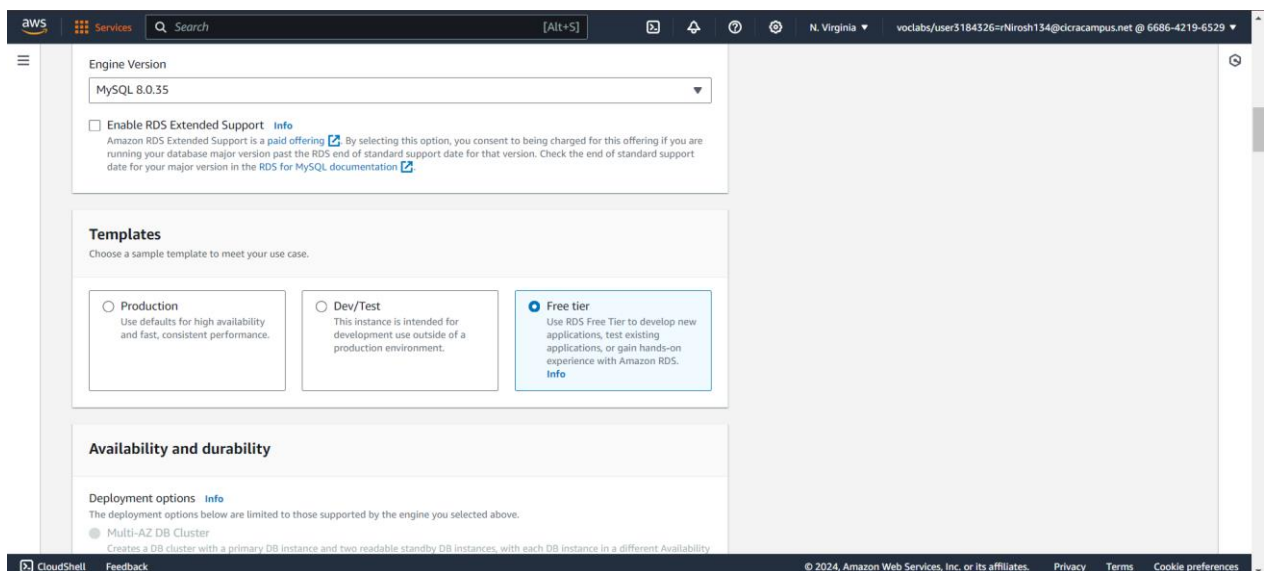
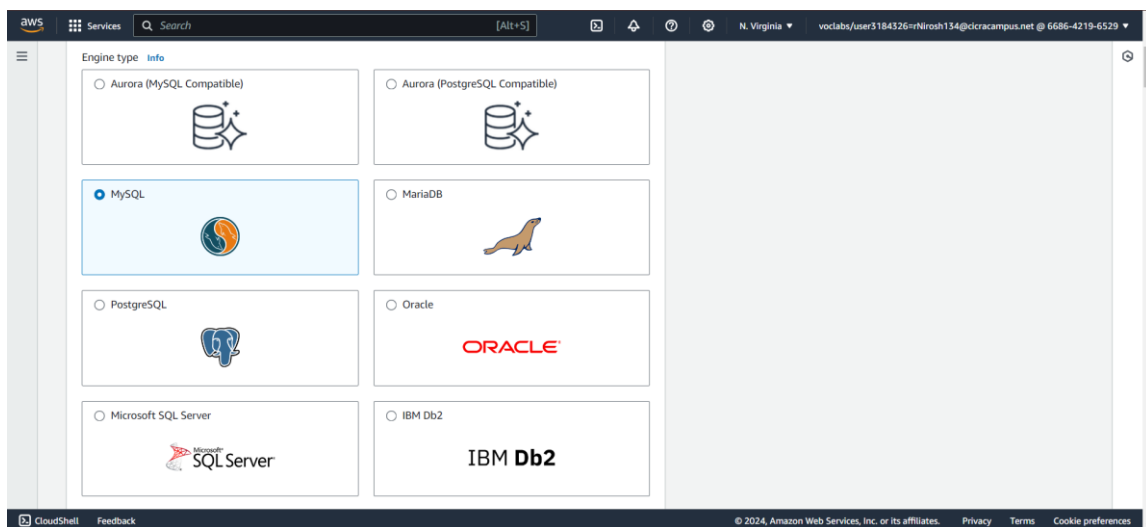
Routes (2) Both ▾ Edit routes

Filter routes

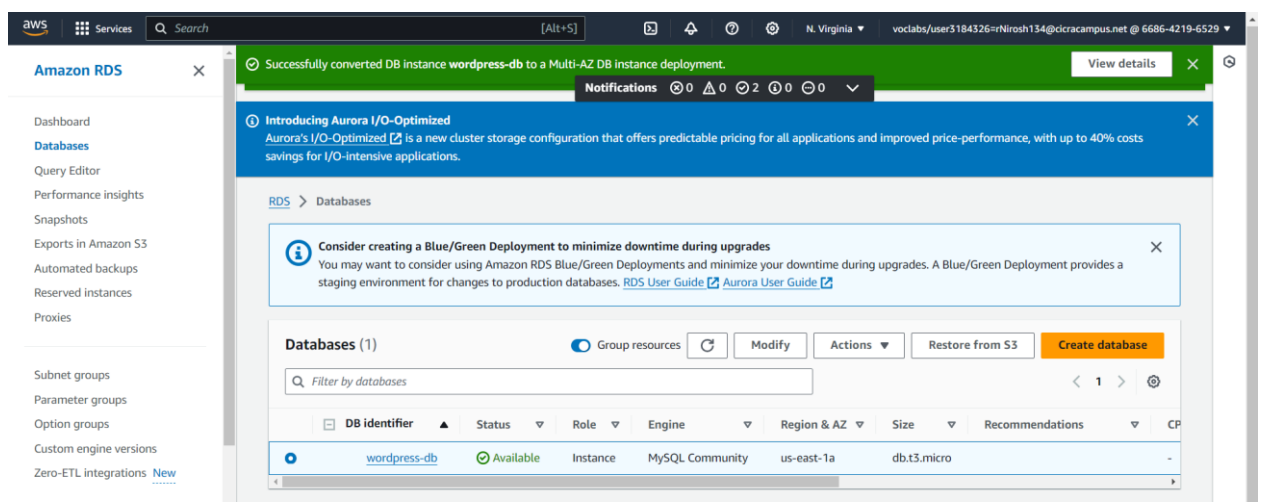
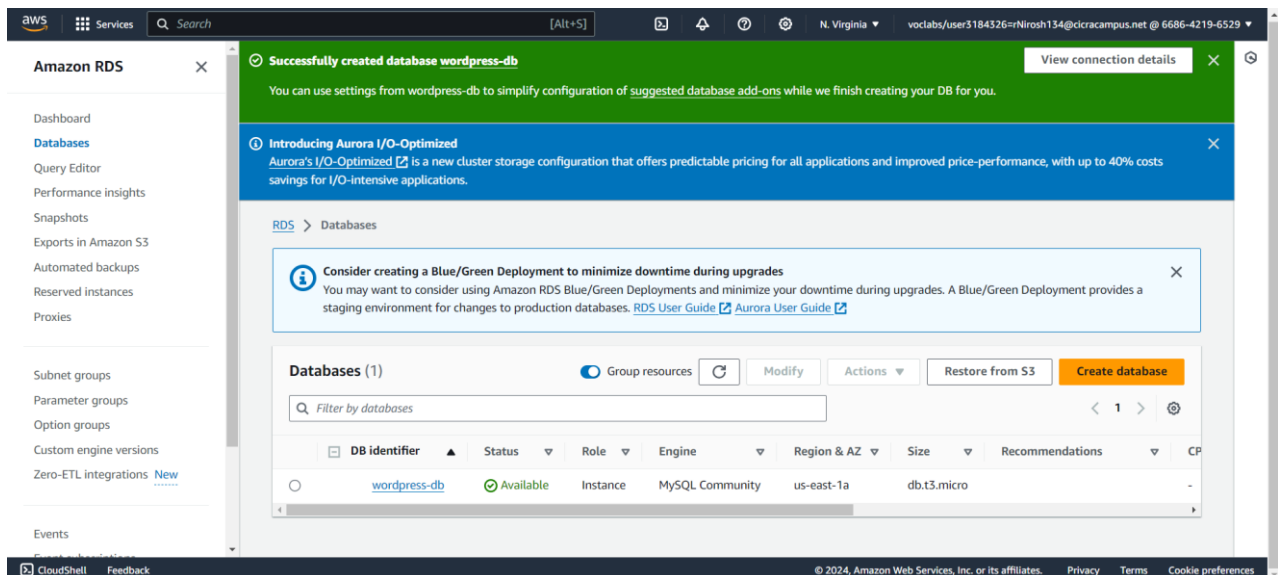
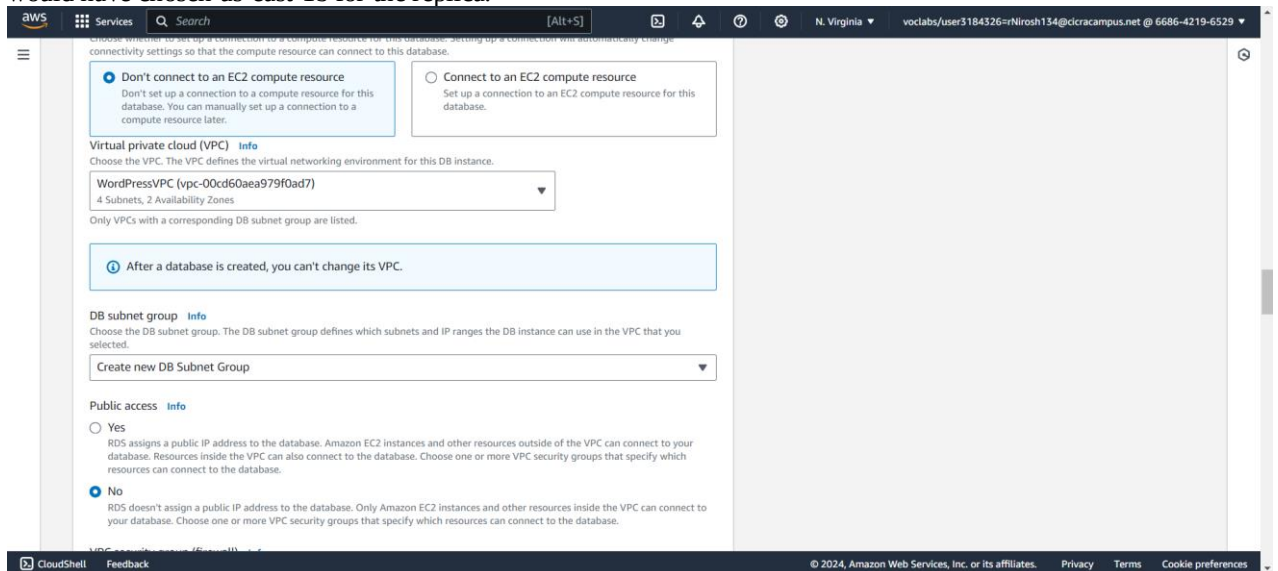
Destination	Target	Status	Propagated
0.0.0.0/0	nat-0299cf2ec72fa83f6	Active	No
192.168.0.0/16	local	Active	No

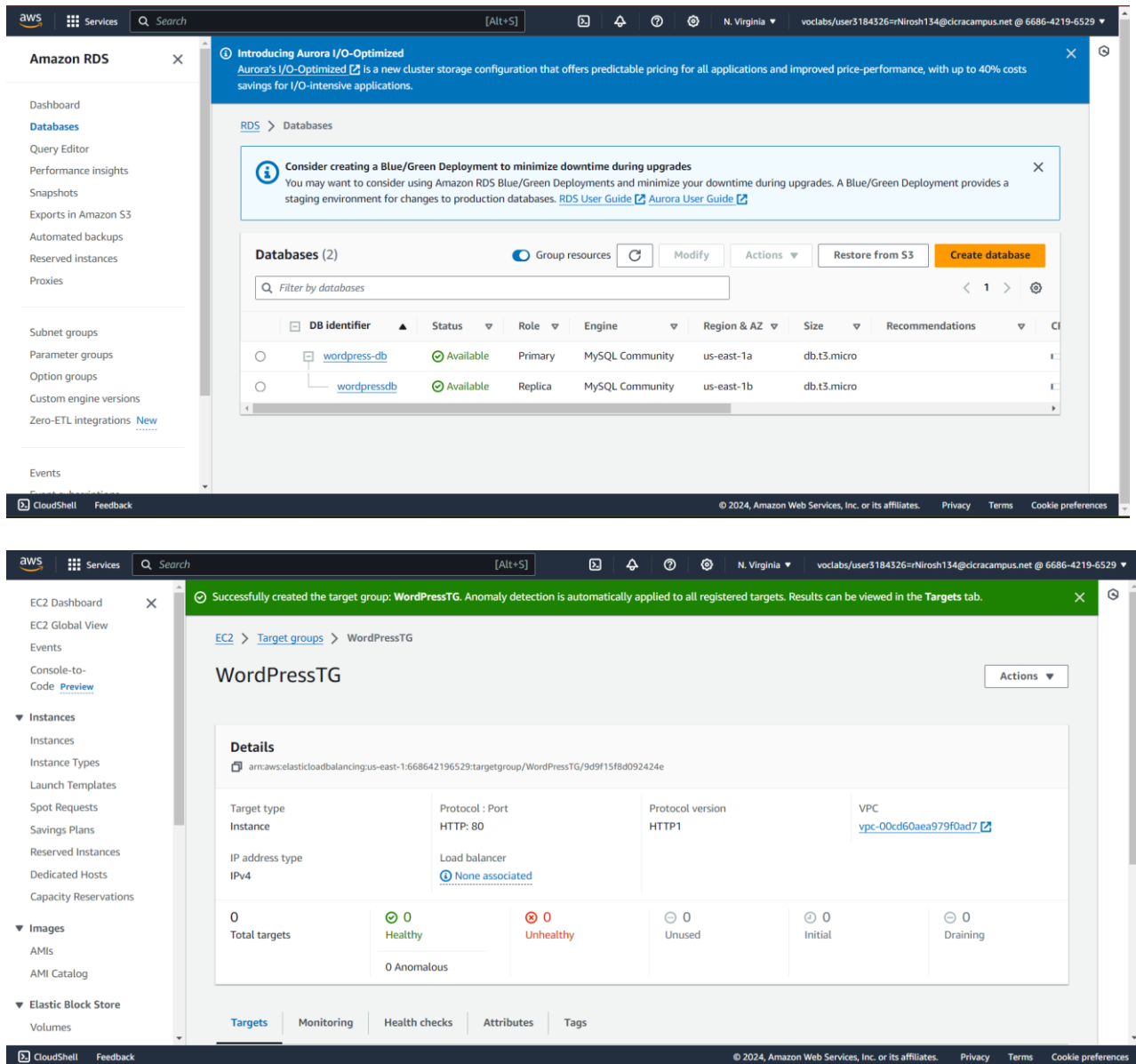
F. Setup RDS

I had to set up a MySQL RDS instance with certain specifications in order to set up the database. After logging into the AWS Management Console, I used the Services menu to see the RDS Dashboard. I selected "Create database" after selecting "Databases" from the left navigation bar. I went with the free tier template, MySQL, and the regular create approach. I gave the instance of the database the name wordpress-db, set the master username to admin, and entered a strong password while configuring the database. I chose the instance class to be db.t3.micro and set aside 20 GB of general purpose (SSD) storage. I used PrivateSubnet1 and PrivateSubnet2 to build a new DB Subnet Group and put WordPressVPC as the VPC in the advanced settings.



I made sure that all public access was blocked and made the wordpress-rds-sg security group, which permits MySQL/Aurora traffic on port 3306. I activated automated backups and entered WordPress as the database name. To increase the database's availability, I turned on Multi-AZ deployment under the availability and durability area. After the database was established, I picked the wordpress-db instance, clicked "Actions," and then selected "Create read replica." I selected db.t3.micro as the instance class and put the read replica in a different Availability Zone than the primary instance—for example, if the primary was in us-east-1a, I would have chosen us-east-1b for the replica.

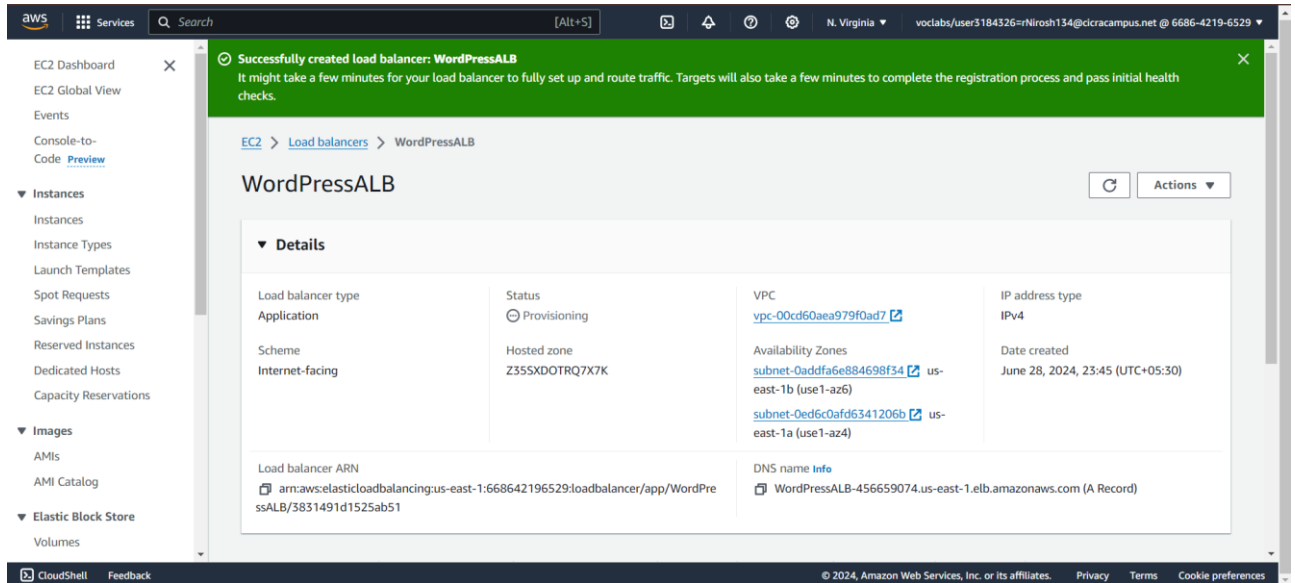




G. Configure ELB

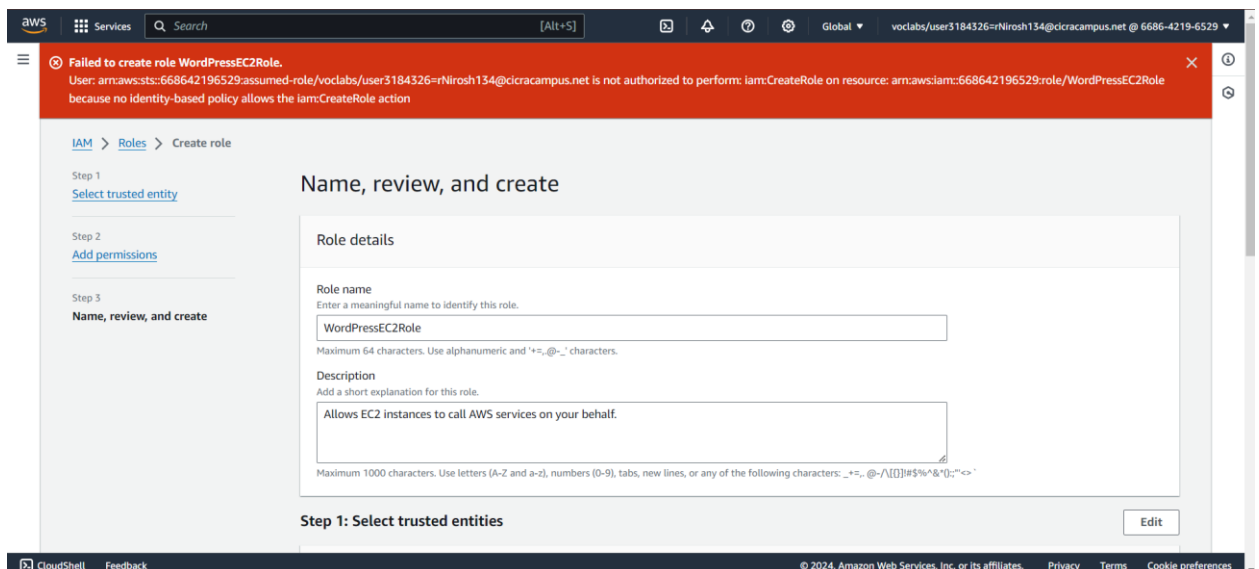
I configured an Application Load Balancer (ALB) to control the distribution of traffic among my web servers. I went to the EC2 Dashboard after logging into the AWS Management Console. I selected "Load Balancers" from the left navigation pane, followed by "Create Load Balancer". I chose the "Application Load Balancer" option, gave it the name WordPressALB, and configured the scheme to use IPv4 addresses for internet access. The load balancer is set up to listen on port 80. I put the ALB in the WordPressVPC's PublicSubnet1 and PublicSubnet2.

I built the wordpress-alb-sg security group in order to permit HTTP traffic on port 80 from the internet. I chose Instance as the target type, set the protocol to HTTP on port 80, and made a new target group called WordPressTG for routing. I registered targets after specifying these parameters because I would be adding the EC2 instances later.



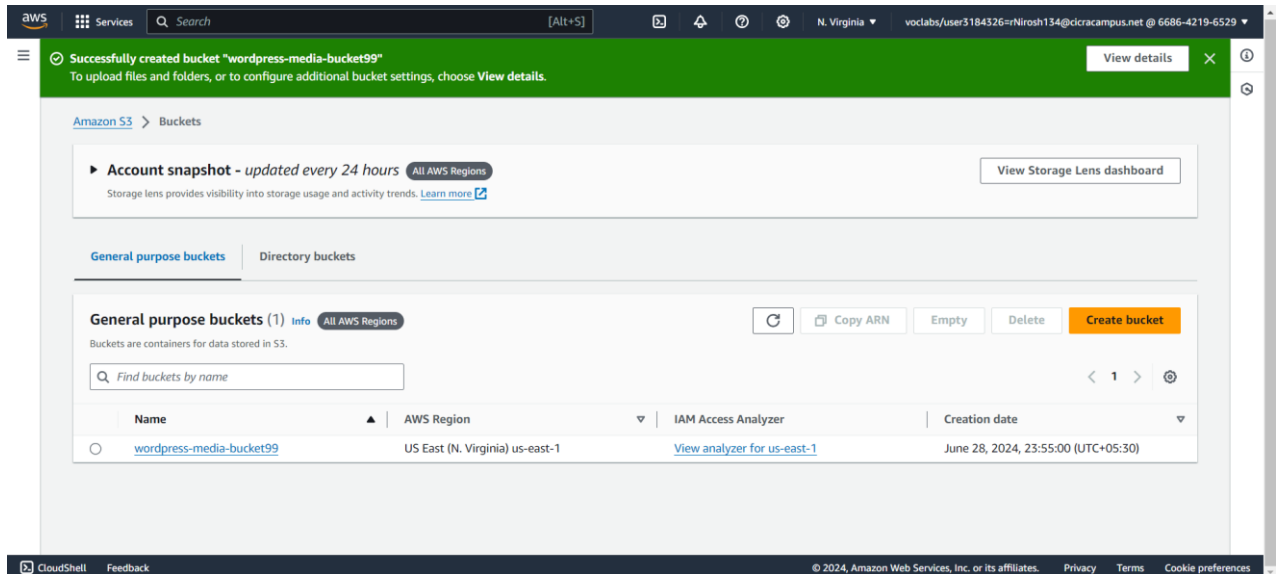
H. Create IAM Role

An error claiming that the IAM role could not be established appeared when I tried to create one. Because the creation of the IAM role is essential for allocating the required rights and roles to the resources in the VPC, this issue stopped me from moving forward with the project.



I. S3 Bucket Setup

I logged into the AWS Management Console and went to the Services menu to get the S3 Dashboard for the S3 bucket. I selected "Create bucket" and gave my bucket a catchy name, like wordpress-media-bucket. I choose the area I wanted and started the creation. I made sure that "Block all public access" was turned on, but I made a note that this option might not be available and that public access might still be enabled by default in the AWS Learner Labs environment.



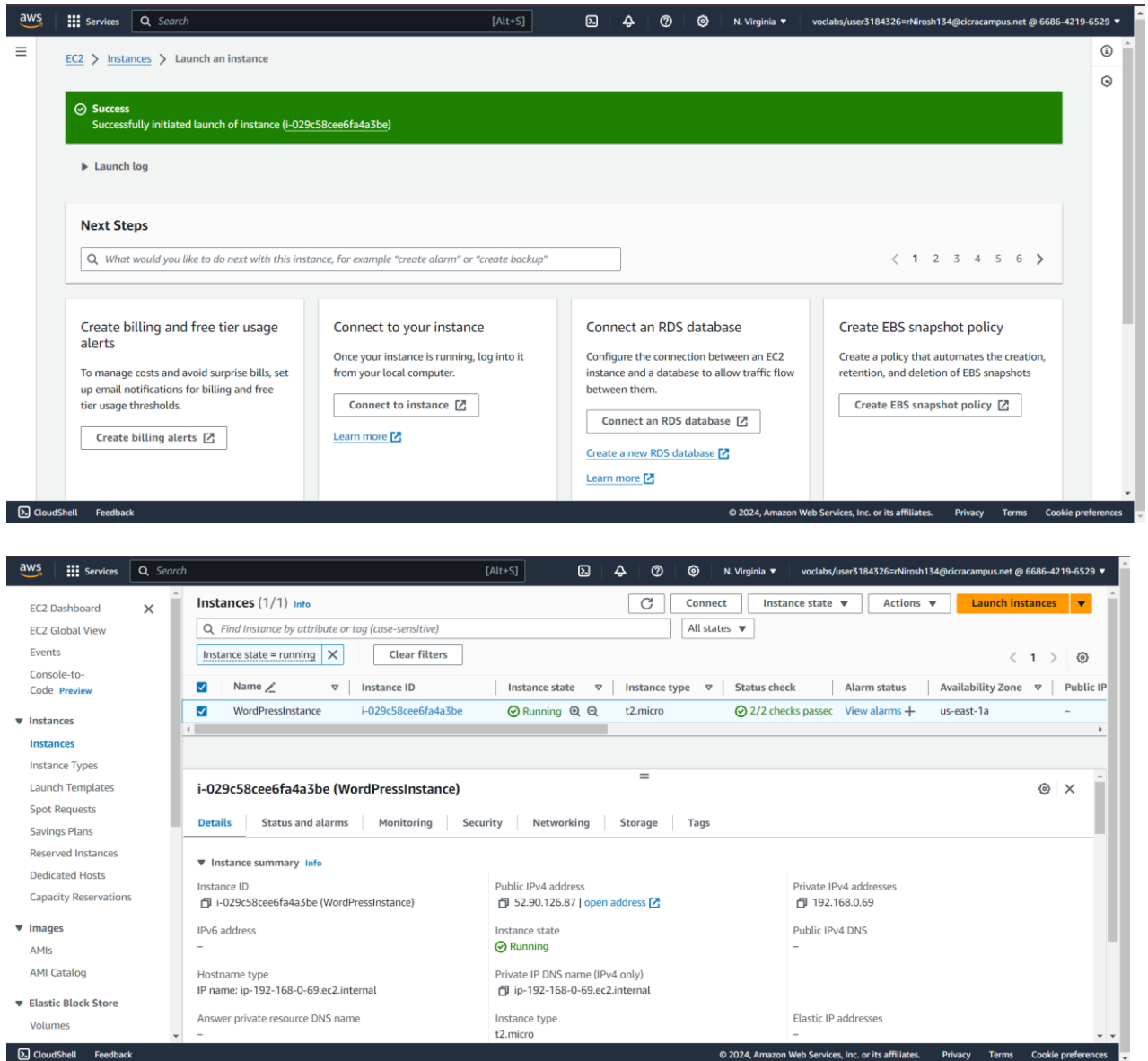
J. Launching EC2 Instance and Configuring WordPress

After that, I started an EC2 instance to host WordPress. I went to the EC2 Dashboard after logging into the AWS Management Console. I selected "Launch instances" after selecting "Instances" in the left navigation pane. I chose the Amazon Linux 2 AMI (HVM), SSD Volume Type for the Amazon Machine Image (AMI). Since the t2.micro instance type is available for free, that's why I went with it. For this EC2 instance, I made a fresh key pair so that I could access SSH later.

I then chose PublicSubnet1 as the subnet and set the instance to use the WordPressVPC. In order to make sure the instance could be accessed from the internet, I turned on Auto-assign Public IP. I opened up a new security group called wordpress-ec2-sg and allowed SSH (port 22) and HTTP (port 80). I tried attaching the WordPressEC2Role that was generated during the IAM setup within the IAM role section. Regretfully, I ran into a problem that prevented me from associating the IAM role here in the prior stage because it could not be created.

To configure the WordPress environment on the EC2 instance, I typed the following user data script in the Advanced Details section:

```
#!/bin/bash
yum -y update
yum -y install php httpd mysql
PHP_VERSION=`php -v | head -n 1 | awk '{print $2}' | awk -F "." '{print $1}`
while [ ${PHP_VERSION} -ne 7 ]
do
amazon-linux-extras install php7.4 -y
PHP_VERSION=`php -v | head -n 1 | awk '{print $2}' | awk -F "." '{print $1}`
done
yum -y install php-mbstring php-xml
wget http://wordpress.org/latest.tar.gz -P /tmp/
tar zxvf /tmp/latest.tar.gz -C /tmp
cp -r /tmp/wordpress/* /var/www/html/
chown apache:apache -R /var/www/html
systemctl enable httpd.service
systemctl start httpd.service
```

I had to wait a few minutes after the instance was launched before the status checks indicated "2/2" and the instance state changed to "running". I opened a browser and went to <http://<EC2-Public-IP>> after that. In order to configure the WordPress website, I would connect it to the RDS instance using the credentials I had already provided. This step should have brought up the WordPress initial setup screen.

But I was unable to finish the EC2 instance's entire setup since I could not create and attach the required IAM role. This stage was not completed because the security group settings and SSH access were not completely functional. As a result, I was unable to complete the project, which included installing WordPress settings on another EC2 instance in PublicSubnet2 and generating an AMI from the server.

IV. DISCUSSION AND REFLECTIONS

The project's goal was to use a variety of AWS services, such as VPC, subnets, RDS, ELB, S3, and EC2, to put up a WordPress website. The VPC, subnets, Internet gateway, NAT gateway, and route tables were successfully created in the first stages. In accordance with the project specifications, I was also able to set up the RDS instance, ELB, and S3 bucket. The IAM role creation procedure presented several obstacles for the project, nevertheless, and ultimately hindered the EC2 instance's successful launch and complete configuration. The inability to finish the WordPress setup and related configurations was caused by this problem. Notwithstanding these difficulties, the project offered insightful knowledge about the intricacies and dependencies of AWS services.

This configuration may improve the scalability, dependability, and security of online applications in practical business settings. The project's reflections highlight the significance of careful planning, role-based access control, and the requirement for troubleshooting abilities while utilizing cloud infrastructure to guarantee smooth application deployment and management.

V. CONCLUSION

The internet gateway, route tables, subnets, security groups, and VPC were all successfully created. However, a mistake in establishing the IAM position caused the project to be stopped at step 5, making further progress impossible. Steps 6 and 7 cannot be completed because they depend on the IAM role for security configurations and permissions. This was due to the inability to create the IAM role.

ACKNOWLEDGEMENT

This report adheres to the NCWE 2018 conference's IEEE format template. We would especially like to appreciate the community forums and AWS documentation for their support during this endeavour.

My video link

<https://www.youtube.com/watch?v=Aynk1SnPHSo>